

第2部分 基础篇

第6章 数据结构基础

学习目标

- ☑ 了解双端队列，能用栈进行简单的表达式解析
- ☑ 熟练掌握链表的数组实现及测试方法
- ☑ 掌握对比测试的方法
- ☑ 掌握完全二叉树的数组实现
- ☑ 掌握二叉树的链式表示法和数组表示法
- ☑ 了解动态内存分配和释放方法及其注意事项
- ☑ 理解内存池的作用以及一种简易实现方法
- ☑ 掌握二叉树的先序、后序、中序遍历和层次遍历
- ☑ 掌握图的 DFS 及连通块计数
- ☑ 掌握图的 BFS 及最短路的输出
- ☑ 掌握拓扑排序算法
- ☑ 掌握欧拉回路算法

本章介绍基础数据结构，包括线性表（包括栈、队列、链表）、二叉树和图。尽管这些内容本身并不算“高级”，但却是很多高级内容的基础。如果数据结构基础没有打好，很难设计出正确、高效的算法。

6.1 再谈栈和队列

例题 6-1 并行程序模拟 (Concurrency Simulator, ACM/ICPC World Finals 1991, UVa210)

你的任务是模拟 n 个程序（按输入顺序编号为 $1 \sim n$ ）的并行执行。每个程序包含不超过 25 条语句，格式一共有 5 种：var = constant（赋值）；print var（打印）；lock；unlock；end。

变量用单个小写字母表示，初始为 0，为所有程序公有（因此在一个程序里对某个变量赋值可能会影响另一个程序）。常数是小于 100 的非负整数。

每个时刻只能有一个程序处于运行态，其他程序均处于等待态。上述 5 种语句分别需要 t_1 、 t_2 、 t_3 、 t_4 、 t_5 单位时间。运行态的程序每次最多运行 Q 个单位时间（称为配额）。当一个程序的配额用完之后，把当前语句（如果存在）执行完之后该程序会被插入一个等待队列中，然后处理器从队首取出一个程序继续执行。初始等待队列包含按输入顺序排列

的各个程序，但由于 lock/unlock 语句的出现，这个顺序可能会改变。

lock 的作用是申请对所有变量的独占访问。lock 和 unlock 总是成对出现，并且不会嵌套。lock 总是在 unlock 的前面。当一个程序成功执行完 lock 指令之后，其他程序一旦试图执行 lock 指令，就会马上被放到一个所谓的阻止队列的尾部（没有用完的配额就浪费了）。当 unlock 执行完毕后，阻止队列的第一个程序进入等待队列的首部。

输入 $n, t_1, t_2, t_3, t_4, t_5, Q$ 以及 n 个程序，按照时间顺序输出所有 print 语句的程序编号和结果。

【分析】

因为有“等待队列”和“阻止队列”的字眼，本题看上去是队列的一个简单应用，但请注意这句话：“阻止队列的第一个程序进入等待队列的首部”。这违反了队列的规则：新元素插入了队列首部而非尾部。

有两个方法可以解决这个问题：一是放弃 STL 队列，自己写一个支持“首部插入”的“队列”：用两个变量 front 和 rear 代表队列当前首尾下标，则传统的入队和出队分别是 $q[++rear] = x$ 和 $x = q[front++]$ ，而“插入到队首”则是 $q[--front] = x$ 。细心的读者应该已经发现：如果 $front=0$ ，则“插入到队首”会产生越界错误。确实如此，不过好在本题不会出现这样的情况（想一想，为什么）。

第二种方法是使用 STL 中的“双端队列”deque。它可以支持快速地在首尾两端进行插入和删除，有兴趣的读者可以自行查阅 STL 文档。

提示 6-1：如果要在“队列”两端进行插入和删除，可以用 STL 中的双端队列 deque。

例题 6-2 铁轨 (Rails, ACM/ICPC CERC 1997, UVa 514)

某城市有一个火车站，铁轨铺设如图 6-1 所示。有 n 节车厢从 A 方向驶入车站，按进站顺序编号为 $1 \sim n$ 。你的任务是判断是否能让它们按照某种特定的顺序进入 B 方向的铁轨并驶出车站。例如，出栈顺序 (5 4 1 2 3) 是不可能的，但 (5 4 3 2 1) 是可能的。

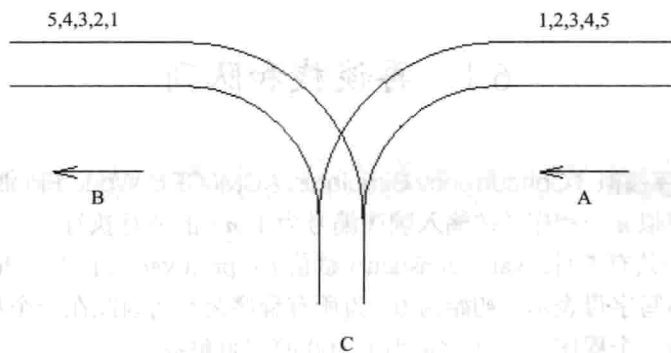


图 6-1 铁轨

为了重组车厢，你可以借助中转站 C。这是一个可以停放任意多节车厢的车站，但由于末端封顶，驶入 C 的车厢必须按照相反的顺序驶出 C。对于每个车厢，一旦从 A 移入 C，就不能再回到 A 了；一旦从 C 移入 B，就不能回到 C 了。换句话说，在任意时刻，只有两种选择： $A \rightarrow C$ 和 $C \rightarrow B$ 。

【分析】

在中转站 C 中, 车厢符合后进先出的原则, 因此是一个栈。代码如下:

```
#include<cstdio>
#include<stack>
using namespace std;
const int MAXN = 1000 + 10;

int n, target[MAXN];

int main(){
    while(scanf("%d", &n) == 1){
        stack<int> s;
        int A = 1, B = 1;
        for(int i = 1; i <= n; i++){
            scanf("%d", &target[i]);
            int ok = 1;
            while(B <= n){
                if(A == target[B]){ A++; B++; }
                else if(!s.empty() && s.top() == target[B]){ s.pop(); B++; }
                else if(A <= n) s.push(A++);
                else { ok = 0; break; }
            }
            printf("%s\n", ok ? "Yes" : "No");
        }
        return 0;
    }
}
```

栈对于表达式求值有着特殊的作用。下面举一例。

例题 6-3 矩阵链乘 (Matrix Chain Multiplication, UVa 442)

输入 n 个矩阵的维度和一些矩阵链乘表达式, 输出乘法的次数。如果乘法无法进行, 输出 error。假定 A 是 $m*n$ 矩阵, B 是 $n*p$ 矩阵, 那么 AB 是 $m*p$ 矩阵, 乘法次数为 $m*n*p$ 。如果 A 的列数不等于 B 的行数, 则乘法无法进行。

例如, A 是 $50*10$ 的, B 是 $10*20$ 的, C 是 $20*5$ 的, 则 $(A(BC))$ 的乘法次数为 $10*20*5$ (BC 的乘法次数) + $50*10*5$ ($(A(BC))$ 的乘法次数) = 3500。

【分析】

本题的关键是解析表达式。本题的表达式比较简单, 可以用一个栈来完成: 遇到字母时入栈, 遇到右括号时出栈并计算, 然后结果入栈。因为输入保证合法, 括号无须入栈。

提示 6-2: 简单的表达式解析可以借助栈来完成。

这里直接给出代码, 其中的道理请读者细细体会。

```
#include<cstdio>
#include<stack>
#include<iostream>
#include<string>
using namespace std;

struct Matrix {
    int a, b;
    Matrix(int a=0, int b=0):a(a),b(b) {}
} m[26];

stack<Matrix> s;

int main() {
    int n;
    cin >> n;
    for(int i = 0; i < n; i++) {
        string name;
        cin >> name;
        int k = name[0] - 'A';
        cin >> m[k].a >> m[k].b;
    }
    string expr;
    while(cin >> expr) {
        int len = expr.length();
        bool error = false;
        int ans = 0;
        for(int i = 0; i < len; i++) {
            if(isalpha(expr[i])) s.push(m[expr[i] - 'A']);
            else if(expr[i] == '(') {
                Matrix m2 = s.top(); s.pop();
                Matrix m1 = s.top(); s.pop();
                if(m1.b != m2.a) { error = true; break; }
                ans += m1.a * m1.b * m2.b;
                s.push(Matrix(m1.a, m2.b));
            }
        }
        if(error) printf("error\n"); else printf("%d\n", ans);
    }
    return 0;
}
```